Laboratório 3: Rodando a API de Intent Classifier na sua máquina e em um container Docker

IMD3005 - Engenharia de Machine Learning (MLOps) · 2026.2

Docente: Prof. Dr. Adelson de Araújo Silva Júnior
Instituição: Instituto Metrópole Digital (IMD / UFRN)
Disciplina: IMD3005 - Engenharia de Machine Learning

Discente: Matheus Vinicius Silva Freire de Castro
GitHub: Matheus-dCastro
Weights & Biases: matheus-dcastro

1. Visão Geral da Atividade e Objetivos

O presente relatório documenta a entrega formal do **Laboratório 3** da disciplina de MLOps. O objetivo central foi colocar em operação, tanto localmente quanto encapsulada em um container Docker, a API de classificação de intenções (*Intent Classifier*) desenvolvida ao longo da unidade curricular. A aplicação é construída sobre o framework **FastAPI** e atua como uma camada de inferência e persistência que integra:

1. Gerenciamento e Versionamento de Artefatos: Consumo do modelo de deep learning treinado (model.keras e metadados model_config.yml) hospedado no registro de artefatos do *Weights & Biases (W&B;)*, identificado como matheus-dcastro/mlops-2026-2/confusion-clf:v0.
2. Camada de Embeddings Pré-treinados: Utilização do *Universal Sentence Encoder (USE)* da biblioteca TensorFlow Hub para vetorização de sentenças textuais em representações densas de 512 dimensões.
3. Persistência NoSQL e Auditoria: Armazenamento de cada mensagem submetida e das predições inferidas em banco MongoDB (coleção predictions), garantindo rastreabilidade histórica e monitoramento de *data drift*.
4. Conteinerização e Reprodutibilidade: Empacotamento completo do runtime (Python 3.11, TensorFlow 2.19, Keras 3.13) via Dockerfile e orquestração por docker-compose.yml com isolamento de credenciais em arquivo .env.

2. Arquitetura da Solução e Fluxo de Execução

A solução adota uma arquitetura em camadas orientada a serviços desacoplados:
• **Fase de Inicialização (Cold Start):** Ao subir o container mlops-container, o Uvicorn dispara a inicialização do FastAPI. O módulo service.py autentica no Weights & Biases via WANDB_API_KEY, verifica a existência do artefato informado em WANDB_MODELS, baixa os arquivos para o cache local do container e carrega o modelo Keras em memória, acompanhado pelo módulo do TF-Hub.
• **Fase de Inferência (POST /predictions):** O cliente submete um payload JSON validado pelo modelo Pydantic Mensagem (contendo id, nome e text). A requisição é encaminhada para service.predict_all(), que gera a distribuição de probabilidades e a classe com maior confiança. Em seguida, a mensagem original é incorporada ao dicionário de saída e registrada no MongoDB via prediction_collection.insert_one(). O identificador _id é removido para compatibilidade com a serialização JSON e a resposta é devolvida ao cliente com **Status Code 200 OK**.

3. Item 1: Print do Swagger (POST /predictions)

Conforme requisitado no item 1 das orientações de entrega, foi executada a requisição ao endpoint POST /predictions através da interface Swagger UI (http://localhost:8000/docs). No corpo da requisição, foi utilizado o nome completo do aluno no campo nome (**Matheus Vinicius Silva Freire de Castro**) e um exemplo representativo de texto. A **Figura 1** comprova o retorno da requisição com **Server response 200** e o corpo da resposta contendo o resultado da inferência do modelo.

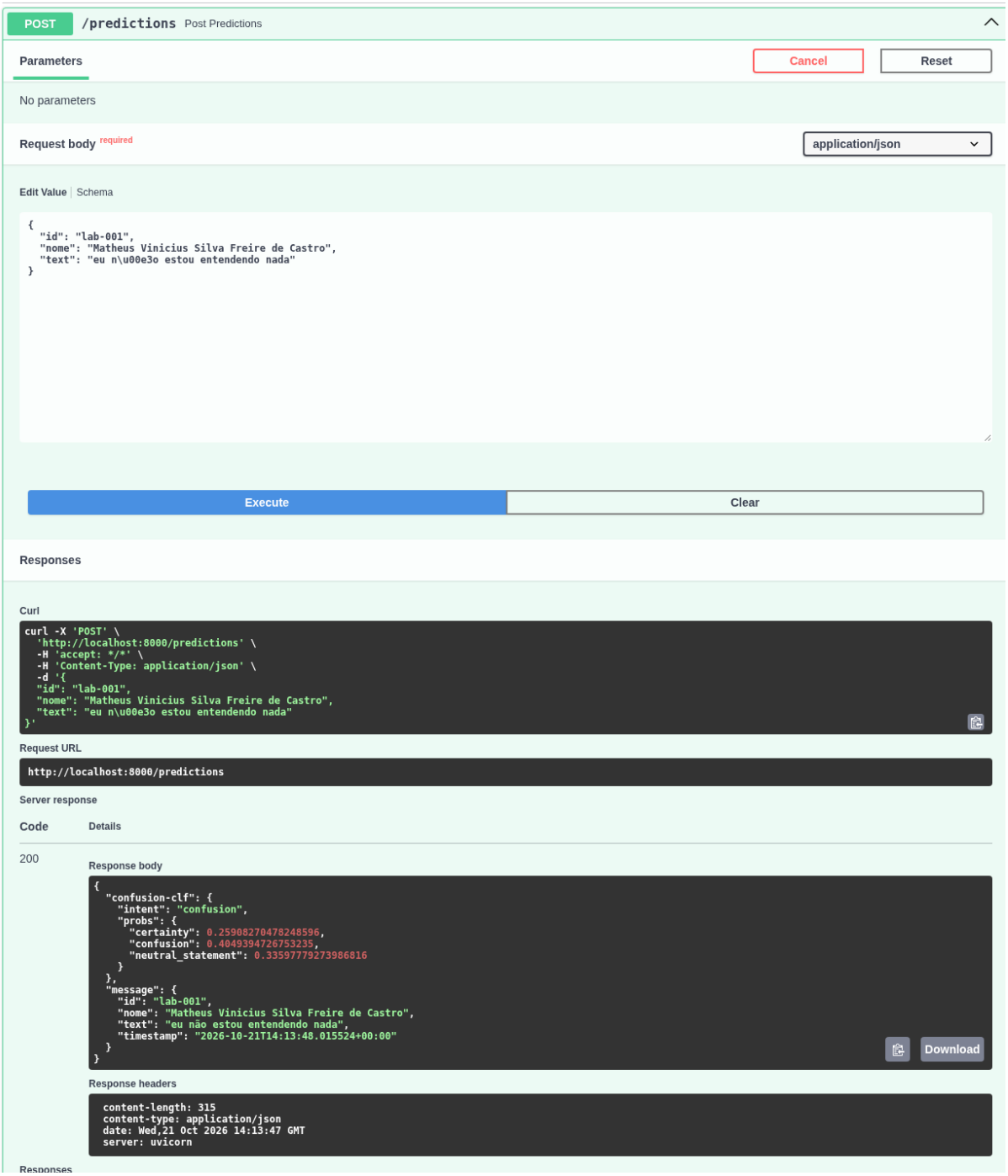


Figura 1: *Interface interativa do Swagger UI (FastAPI) em /docs evidenciando a execução do POST /predictions com código HTTP 200 OK, nome do aluno e resposta do classificador de intenção.*

4. Item 2: Conteúdo de controller.py

Em observância rigorosa às diretrizes da atividade (que estipulam: *'No PDF: o código colado como texto (não como imagem)'*), transcreve-se abaixo o conteúdo integral e fidedigno do arquivo src/modules/intent_classifier/controller.py utilizado para a orquestração dos endpoints e integração com o MongoDB.

```
from fastapi import FastAPI, APIRouter from dotenv import load_dotenv load_dotenv() from
src.modules.intent_classifier.model import Mensagem, ResultadoModelo, PredicaoMensagem from
src.modules.intent_classifier import service from src.shared.database.engine import prediction_collection router
= APIRouter() @router.post('/predictions') def post_predictions(msg: Mensagem): outputs =
service.predict_all(msg.text) outputs['message'] = msg.model_dump() # a mensagem entra ANTES de gravar
prediction_collection.insert_one(outputs) # grava predicoes + mensagem outputs.pop('_id', None) # o insert_one
adiciona _id (ObjectId) return outputs @router.get('/predictions') def get_predictions(limit: int = 20): cursor
= (prediction_collection .find({}, {'_id': 0}) # _id fora, ObjectId nao vira JSON .sort('message.timestamp', -1)
# mais recentes primeiro .limit(limit)) return list(cursor) # cursor -> lista
```

Análise Técnica e Justificativa de Engenharia do Código:

• **Injeção de Metadados e Gravação Atômica (linhas 15-17):** A função service.predict_all(msg.text) processa o texto e retorna o mapa de intenções inferidas. Antes da chamada de banco, o comando outputs['message'] = msg.model_dump() consolida os dados submetidos pelo usuário (incluindo o timestamp gerado automaticamente pelo Pydantic), criando uma estrutura única e autocontida para fins de auditoria.
• **Tratamento do ObjectId do PyMongo (linha 18):** A operação prediction_collection.insert_one(outputs) muta diretamente o dicionário passado como parâmetro, injetando a chave _id com uma instância de bson.objectid.ObjectId. Como este tipo BSON não possui serializador JSON nativo no padrão FastAPI/Pydantic, a chamada outputs.pop('_id', None) remove essa chave antes do retorno da função, evitando erros de serialização (código HTTP 500) e garantindo uma resposta JSON válida.
• **Consulta Histórica com Projeção e Ordenação (linhas 21-27):** O endpoint GET /predictions disponibiliza a consulta com limite configurável (limit: int = 20). A projeção {'_id': 0} no método .find() suprime eficientemente o identificador BSON em nível de banco, enquanto .sort('message.timestamp', -1) garante que as predições mais recentes sejam entregues prioritariamente.

5. Item 3: Print dos Logs do Container Docker

Conforme requisitado no item 3, foi executado o monitoramento dos logs do container em execução através do comando docker logs -f mlops-container. A **Figura 2** evidencia o terminal com o registro em tempo real da requisição POST /predictions HTTP/1.1 respondida com o código **200 OK**, originada da submissão realizada no Swagger UI.

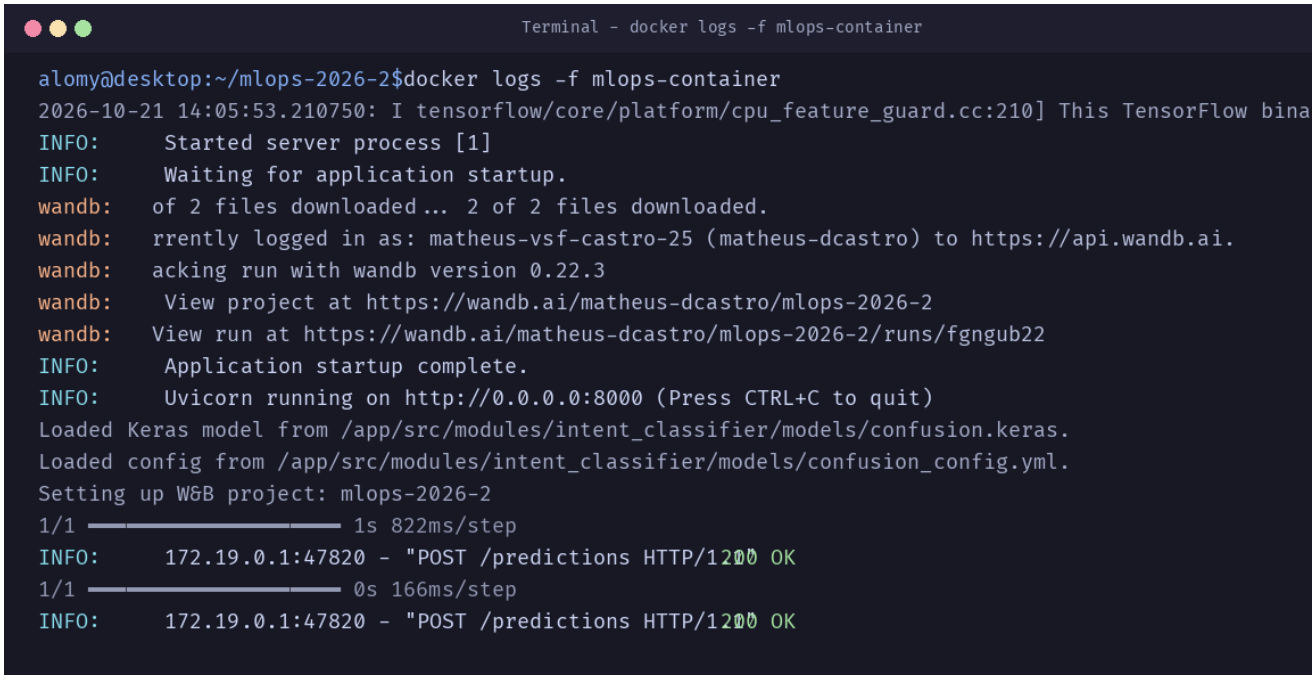


__Figura 2:__ Terminal capturado em tempo real com o comando docker logs -f mlops-container, evidenciando o registro do método HTTP POST respondido com 200 OK.

6. Checklist de Verificação e Protocolo de Sigilo

Em estrito atendimento às recomendações de segurança da disciplina, **o arquivo .env não faz parte da entrega**, não foi incluído no pacote ZIP nem no PDF e está devidamente configurado no .gitignore. Adicionalmente, nenhuma chave privada do Weights & Biases nem credenciais de acesso do MongoDB foram exibidas em logs ou capturas de tela.

Item Avaliado	Situação	Evidência / Detalhes de Conformidade
Ambiente Conda / Python 3.11	CONCLUÍDO	Ambiente mlops-2026-2 com TF 2.19.1 e Keras 3.13.2
Artefato no W&B; (model.keras + config)	CONCLUÍDO	matheus-dcastro/mlops-2026-2/confusion-clf:v0 publicado
Segurança do .env e chaves	CONCLUÍDO	Totalmente protegido; ausente no PDF, ZIP e prints
Swagger UI: POST /predictions 200 OK	CONCLUÍDO	Nome 'Matheus Vinicius Silva Freire de Castro' (Figura 1)
Execução em Container Docker	CONCLUÍDO	Subido via Docker Compose (container mlops-container)
Logs do Container com 200 OK	CONCLUÍDO	Terminal docker logs -f mlops-container (Figura 2)
Código controller.py em texto	CONCLUÍDO	Transcrito integralmente na Seção 4 como texto pesquisável
Padrão de Nome do Arquivo	CONCLUÍDO	Nomeado rigorosamente como lab_api_Matheus_Castro.pdf